Cognizant digital nurture hands on exercises – WEEK 1

Design principles and Patterns-Hands on ---Exercise 1: Implementing the Singleton Pattern

Step 1: Create the Logger class

```
        public class Logger {
    // Step 1: Create a private static instance
    private static Logger instance;
    // Step 2: Private constructor
    private Logger() {
        System.out.println("Logger instance created.");
    }
    // Step 3: Public static method to return the single instance
    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }
    // Logging method
    public void log(String message) {
        System.out.println("Log: " + message);
    }
}
```

Step 2: Create the Test Class

SingletonTest.java

```
public class SingletonTest {
    public static void main(String[] args) {
        // Get Logger instances
        Logger logger1 = Logger.getInstance();
        Logger logger2 = Logger.getInstance();
        // Use the logger
        logger1.log("Application Started");
        logger2.log("Loading Data");

        // Check if both references point to the same object
        if (logger1 == logger2) {
            System.out.println("Only one Logger instance exists.");
        } else {
            System.out.println("Multiple Logger instances exist.");
        }

        // Print hash codes
        System.out.println("Logger1 HashCode: " + logger1.hashCode());
        System.out.println("Logger2 HashCode: " + logger2.hashCode());
    }
}
```

Expected Output:

Logger instance created.

Log: Application Started

Log: Loading Data

Only one Logger instance exists.

Logger1 HashCode: 366712642

Logger2 HashCode: 366712642

EXERCISE 2: Implementing the Factory Method Pattern

Step 1: Create the Document Interface

Document.java

```
public interface Document {
   void open();
}
```

Step 2: Create Concrete Document Classes

WordDocument.java

```
public class WordDocument implements Document {

   @Override
   public void open() {
      System.out.println("Word Document is opened.");
   }
}
//PdfDocument.java
public class PdfDocument implements Document {

   @Override
   public void open() {
      System.out.println("PDF Document is opened.");
   }
}
```

```
//ExcelDocument.java
public class ExcelDocument implements Document {
  @Override
  public void open() {
    System.out.println("Excel Document is opened.");
  }
}
```

Step 3: Create the Abstract Factory Class

DocumentFactory.java

```
public abstract class DocumentFactory {
  public abstract Document createDocument();
}
```

Step 4: Create Concrete Factory Classes

WordDocumentFactory.java

```
public class WordDocumentFactory extends DocumentFactory {
  @Override
  public Document createDocument() {
    return new WordDocument();
  }
}
```

PdfDocumentFactory.java

```
public class PdfDocumentFactory extends DocumentFactory {

  @Override
  public Document createDocument() {
    return new PdfDocument();
  }
}
```

ExcelDocumentFactory.java

```
public class ExcelDocumentFactory extends DocumentFactory {

  @Override
  public Document createDocument() {
    return new ExcelDocument();
  }
}
```

Step 5: Create the Test Class

FactoryMethodTest.java

```
public class FactoryMethodTest {

  public static void main(String[] args) {

    // Create Word Document
    DocumentFactory wordFactory = new WordDocumentFactory();
    Document word = wordFactory.createDocument();
    word.open();

    // Create PDF Document
    DocumentFactory pdfFactory = new PdfDocumentFactory();
    Document pdf = pdfFactory.createDocument();
    pdf.open();

    // Create Excel Document
    DocumentFactory excelFactory = new ExcelDocumentFactory();
    Document excel = excelFactory.createDocument();
    excel.open();
  }
}
```

Expected Output

Word Document is opened.
PDF Document is opened.
Excel Document is opened.

Project Structure

FactoryMethodPatternExample
Document.java
Creating Word Document...

Word Document is opened.

Creating PDF Document...

PDF Document is opened.

Creating Excel Document...

Excel Document is opened.

Data Structures -Hands on ----Exercise 1: E-commerce Platform Search Function

Product.java

```
public class Product {
  int productId;
  String productName;
  String category;

  public Product(int productId, String productName, String category) {
    this.productId = productId;
    this.productName = productName;
    this.category = category;
```

```
    }

    public void display() {
        System.out.println(productId + " " + productName + " " + category);
    }
}
```

---

3. Implementation

SearchExample.java

```
import java.util.Arrays;
import java.util.Comparator;

public class SearchExample {

    // Linear Search
    public static Product linearSearch(Product[] products, int id) {
        for (Product product : products) {
            if (product.productId == id) {
                return product;
            }
        }
        return null;
    }

    // Binary Search
    public static Product binarySearch(Product[] products, int id) {
        int low = 0;
        int high = products.length - 1;

        while (low <= high) {
            int mid = (low + high) / 2;
```

```
            if (products[mid].productId == id) {
                return products[mid];
            } else if (products[mid].productId < id) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return null;
    }

    public static void main(String[] args) {

        Product[] products = {
            new Product(103, "Keyboard", "Electronics"),
            new Product(101, "Laptop", "Electronics"),
            new Product(104, "Mouse", "Electronics"),
            new Product(102, "Shoes", "Fashion")
        };

        // Linear Search
        Product result1 = linearSearch(products, 102);

        if (result1 != null) {
            System.out.println("Linear Search Result:");
            result1.display();
        } else {
            System.out.println("Product not found.");
        }

        // Sort array for Binary Search
        Arrays.sort(products, Comparator.comparingInt(p -> p.productId));
```

```
        Product result2 = binarySearch(products, 102);

        if (result2 != null) {
            System.out.println("Binary Search Result:");
            result2.display();
        } else {
            System.out.println("Product not found.");
        }
    }
}
```

---

4. Output

Linear Search Result:
102 Shoes Fashion

Binary Search Result:
102 Shoes Fashion

Exercise 2: Financial Forecasting

1. Understand Recursive Algorithms

Recursion

Recursion is a programming technique in which a method calls itself to solve a problem. A recursive algorithm breaks a problem into smaller subproblems until it reaches a base case, which stops further recursive calls. It simplifies problems that can be divided into similar smaller tasks.

2.Setup

FutureValue.java

```
public class FutureValue {

   // Recursive method to calculate future value
   public static double futureValue(double presentValue, double
growthRate, int years) {

      if (years == 0) {
         return presentValue;
      }

      return futureValue(presentValue, growthRate, years - 1) * (1 +
growthRate);
   }

   public static void main(String[] args) {

      double presentValue = 10000;
      double growthRate = 0.10; // 10%
      int years = 5;

      double result = futureValue(presentValue, growthRate, years);

      System.out.println("Future Value after " + years + " years = " +
result);
   }
}
```

---

3. Implementation

The recursive algorithm calculates the future value using the formula:

$$\text{Future Value} = \text{Present Value} \times (1 + \text{Growth Rate})^{\text{Years}}$$

Each recursive call reduces the number of years by one until it reaches the base case (years == 0).

---

4. Output

Future Value after 5 years = 16105.1

5. Analysis

Time Complexity

- **Time Complexity: O(n)**, where **n** is the number of years.
- **Space Complexity: O(n)** due to the recursive call stack.

Optimization

The recursive solution can be optimized by:

- **Memoization:** Store previously computed values to avoid repeated calculations.
- **Iteration:** Use a loop instead of recursion to eliminate the recursive call stack and reduce memory usage.
- **Direct Formula:** Use the mathematical formula:

$$\text{Future Value} = \text{Present Value} \times (1 + \text{Growth Rate})^{\text{Years}}$$

SQL-HANDS ON -EXERCISE 1: Control Structures

Scenario 1: Apply 1% Discount to Loan Interest Rates for Customers Above 60 Years

PL/SQL Code

```
DECLARE
BEGIN
   FOR rec IN (
```

```
    SELECT CustomerID, Age
    FROM Customers
    WHERE Age > 60
  )
  LOOP
    UPDATE Loans
    SET InterestRate = InterestRate - 1
    WHERE CustomerID = rec.CustomerID;
  END LOOP;

  COMMIT;
  DBMS_OUTPUT.PUT_LINE('1% discount applied successfully.');
END;
/
```

Output

1% discount applied successfully.

Scenario 2: Promote Customers to VIP Status

PL/SQL Code

```
DECLARE
BEGIN
  FOR rec IN (
    SELECT CustomerID, Balance
    FROM Customers
    WHERE Balance > 10000
  )
  LOOP
    UPDATE Customers
    SET IsVIP = 'TRUE'
    WHERE CustomerID = rec.CustomerID;
```

```
  END LOOP;

  COMMIT;
  DBMS_OUTPUT.PUT_LINE('VIP status updated successfully.');
END;
/
```

Output

VIP status updated successfully.

Scenario 3: Send Loan Due Reminders

PL/SQL Code

```
DECLARE
BEGIN
  FOR rec IN (
    SELECT CustomerID, LoanID, DueDate
    FROM Loans
    WHERE DueDate BETWEEN SYSDATE AND SYSDATE + 30
  )
  LOOP
    DBMS_OUTPUT.PUT_LINE(
      'Reminder: Loan ' || rec.LoanID ||
      ' for Customer ' || rec.CustomerID ||
      ' is due on ' || TO_CHAR(rec.DueDate, 'DD-MON-YYYY')
    );
  END LOOP;
END;
/
```

Sample Output

Reminder: Loan 101 for Customer 1001 is due on 15-JUL-2026
Reminder: Loan 102 for Customer 1005 is due on 22-JUL-2026
Reminder: Loan 103 for Customer 1008 is due on 30-JUL-2026

Exercise 2: Stored Procedures

Scenario 1: Process Monthly Interest for Savings Accounts

Stored Procedure

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
AS
BEGIN
   UPDATE Accounts
   SET Balance = Balance + (Balance * 0.01)
   WHERE AccountType = 'Savings';

   COMMIT;

   DBMS_OUTPUT.PUT_LINE('Monthly interest processed
successfully.');
END;
/
```

Execute Procedure

```
BEGIN
   ProcessMonthlyInterest;
END;
/
```

Output

Monthly interest processed successfully.

Scenario 2: Update Employee Bonus

Stored Procedure

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
   p_Department IN VARCHAR2,
   p_BonusPercent IN NUMBER
)
AS
BEGIN
   UPDATE Employees
   SET Salary = Salary + (Salary * p_BonusPercent / 100)
   WHERE Department = p_Department;

   COMMIT;

   DBMS_OUTPUT.PUT_LINE('Employee bonus updated successfully.');
END;
/
```

Execute Procedure

```
BEGIN
   UpdateEmployeeBonus('Finance',10);
END;
/
```

Output

Employee bonus updated successfully.

Scenario 3: Transfer Funds Between Accounts

Stored Procedure

```
CREATE OR REPLACE PROCEDURE TransferFunds(
   p_FromAccount IN NUMBER,
```

```
  p_ToAccount IN NUMBER,
  p_Amount IN NUMBER
)
AS
  v_Balance NUMBER;
BEGIN
  SELECT Balance
  INTO v_Balance
  FROM Accounts
  WHERE AccountID = p_FromAccount;

  IF v_Balance >= p_Amount THEN

    UPDATE Accounts
    SET Balance = Balance - p_Amount
    WHERE AccountID = p_FromAccount;

    UPDATE Accounts
    SET Balance = Balance + p_Amount
    WHERE AccountID = p_ToAccount;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Funds transferred successfully.');

  ELSE
    DBMS_OUTPUT.PUT_LINE('Insufficient balance.');
  END IF;
END;
/
```

Execute Procedure

```
BEGIN
  TransferFunds(101,102,5000);
END;
/
```

Output (Successful Transfer)

Funds transferred successfully.

Output (Insufficient Balance)

Insufficient balance.

J UNIT HANDS ON -EXERCISE 1: Setting Up JUnit

Step 1: Create a New Java Project

Create a new Java project in any IDE such as **IntelliJ IDEA** or **Eclipse**.

Step 2: Add JUnit Dependency

If you are using **Maven**, add the following dependency to your pom.xml file:

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>
```

Step 3: Create a Test Class

Create a test class named **CalculatorTest.java**.

```
import static org.junit.Assert.assertEquals;
import org.junit.Test;
```

```
public class CalculatorTest {

    @Test
    public void testAddition() {
        int result = 10 + 20;
        assertEquals(30, result);
    }
}
```

Step 4: Run the Test

- Right-click on **CalculatorTest.java**.
- Select **Run As → JUnit Test** (Eclipse) or **Run 'CalculatorTest'** (IntelliJ IDEA).

Expected Output

```
JUnit version 4.13.2
.
Time: 0.005

OK (1 test)
```

Exercise 2: Assertions in JUnit

AssertionsTest.java

```
import static org.junit.Assert.*;
import org.junit.Test;

public class AssertionsTest {
```

```
    @Test
    public void testAssertions() {

        // Assert Equals
        assertEquals(5, 2 + 3);

        // Assert True
        assertTrue(5 > 3);

        // Assert False
        assertFalse(5 < 3);

        // Assert Null
        assertNull(null);

        // Assert Not Null
        assertNotNull(new Object());
    }
}
```

Expected Output

```
JUnit version 4.13.2
.
Time: 0.003

OK (1 test)
```

Exercise 2: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in JUnit

Calculator.java

```
public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }
}
```

CalculatorTest.java

```
import static org.junit.Assert.*;
import org.junit.Before;
import org.junit.After;
import org.junit.Test;

public class CalculatorTest {

    private Calculator calculator;

    @Before
    public void setUp() {
        calculator = new Calculator();
        System.out.println("Setup executed");
    }

    @After
    public void tearDown() {
        System.out.println("Teardown executed");
    }

    @Test
    public void testAdd() {

        // Arrange
```

```
        int a = 10;
        int b = 20;

        // Act
        int result = calculator.add(a, b);

        // Assert
        assertEquals(30, result);
    }
}
```

Expected Output

```
Setup executed
Teardown executed

JUnit version 4.13.2
.
Time: 0.004

OK (1 test)
```

Exercise 3: Mocking and Stubbing (Mockito)

ExternalApi.java

```
public interface ExternalApi {
    String getData();
}
```

MyService.java

```
public class MyService {

    private ExternalApi api;

    public MyService(ExternalApi api) {
        this.api = api;
    }

    public String fetchData() {
        return api.getData();
    }
}
```

MyServiceTest.java

```
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.assertEquals;

import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class MyServiceTest {

    @Test
    public void testExternalApi() {

        // Create mock object
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);

        // Stub the method
        when(mockApi.getData()).thenReturn("Mock Data");

        // Use mock object
```

```
        MyService service = new MyService(mockApi);

        String result = service.fetchData();

        // Verify result
        assertEquals("Mock Data", result);
    }
}
```

Expected Output

```
BUILD SUCCESSFUL

Tests run: 1
Failures: 0
Errors: 0
Skipped: 0

Process finished with exit code 0
```

Exercise 4: Verifying Interactions (Mockito)

Service.java

```
public interface Service {
    void sendMessage(String message);
}
```

ServiceTest.java

```
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.Test;
```

```
import org.mockito.Mockito;

public class ServiceTest {

   @Test
   public void testVerifyInteraction() {

      // Create mock object
      Service mockService = Mockito.mock(Service.class);

      // Call the method
      mockService.sendMessage("Hello");

      // Verify the interaction
      verify(mockService).sendMessage("Hello");
   }
}
```

Expected Output

```
BUILD SUCCESSFUL

Tests run: 1
Failures: 0
Errors: 0
Skipped: 0

Process finished with exit code 0
```

EXERCISE 5: BEST TRAVEL DESTINATIONS THIS YEAR

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class TravelDestinations {

   private static final Logger logger =
LoggerFactory.getLogger(TravelDestinations.class);

   public static void main(String[] args) {

      logger.info("Best Travel Destinations This Year:");

      logger.info("1. Japan - Famous for cherry blossoms, culture, and
technology.");
      logger.info("2. Switzerland - Known for beautiful mountains and
scenic landscapes.");
      logger.info("3. Italy - Popular for historical monuments, art, and
delicious cuisine.");
      logger.info("4. Bali, Indonesia - Ideal for beaches, temples, and
relaxation.");
      logger.info("5. Dubai, UAE - Famous for luxury shopping,
skyscrapers, and desert safaris.");

      logger.warn("Always check weather conditions and travel
advisories before planning your trip.");
      logger.error("Travel booking failed due to network connection.
Please try again.");
   }
}
```

Expected Output

```
INFO  - Best Travel Destinations This Year:
INFO  - 1. Japan - Famous for cherry blossoms, culture, and technology.
```

INFO - 2. Switzerland - Known for beautiful mountains and scenic landscapes.
INFO - 3. Italy - Popular for historical monuments, art, and delicious cuisine.
INFO - 4. Bali, Indonesia - Ideal for beaches, temples, and relaxation.
INFO - 5. Dubai, UAE - Famous for luxury shopping, skyscrapers, and desert safaris.
WARN - Always check weather conditions and travel advisories before planning your trip.
ERROR - Travel booking failed due to network connection. Please try again.